Threads

Operating Systems

Universität Innsbruck

Summer Semester 2026

1 Introduction

In this session, you will work with threads by creating and passing arguments to them. You will use the `pthreads` library to do this assignment. When you compile a program that uses this library, you need to indicate your desire to use `pthreads` by adding the `-pthread` option at the end of the compiler command line. This option specifies that the `pthreads` library should be linked.

```
gcc -o myprog myprog.c -pthread
```

You can check the manual pages with the following commands:

```
man 7 pthreads
man 3 pthread_*
```

The `pthread_*` refers to any function in the pthread library, such as `pthread_create` and `pthread_join`.

2 Creating and Destroying Threads

The first step to understanding how to build a multi-threaded program is to understand how to create and destroy threads. To create a new thread, you need to use the `pthread_create()` function. It returns a thread identifier that can be used in subsequent calls (as its passed argument). The second parameter is a pointer to a thread attribute object, and `NULL` means to use the default attributes (suitable for many cases). The third parameter is a pointer to the thread's function to execute, and the final parameter is the argument passed to the thread function. This function returns 0 on successful creation; otherwise, it returns the error number. Your program should then wait for each thread to terminate and collect its results by calling `pthread_join()`, which waits for the thread with the ID as its first argument and collects the thread's return value in its second argument.

```c
#include <pthread.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

void *threadfunction(void *arg) {
    printf("%s\n", (char *)arg);
    return NULL;
}

int main() {
    pthread_t thread;
    char *msg = "Hello world!";
    int createerror = pthread_create(&thread, NULL, threadfunction, (void *) msg);
    pthread_join(thread, NULL);
    return 0;
}
```

Now, you have multiple threads (N), each gets a number and runs in any order. Then the main process waits for them to finish:

```c
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <time.h>

static void* threadfunction(void* arg) {
    long id = (long)arg;

    int delay = rand() % 100000;   // random delay up to 100000 microseconds
    usleep(delay);
    printf("Hello from thread %ld\n", id);
    return NULL;
}

int main(void) {
    const int N = 6;
    pthread_t t[N];

    for (long i = 0; i < N; i++) {
        pthread_create(&t[i], NULL, threadfunction, (void*)i);
    }

    for (int i = 0; i < N; i++) {
        pthread_join(t[i], NULL);
    }

    puts("All done.");
    return 0;
}
```

3 Shared Memory and Stack in Threads

Now you will observe how threads access different data of global (shared) and local data, which represents the addresses of shared data between them and local stack data that is different among them.

```c
#include <pthread.h>
#include <stdio.h>

static int global_counter = 0;

static void* thread_func(void* arg) {
    int local = 42;
    printf("tid=%lu global@%p local@%p\n",
           (unsigned long)pthread_self(),
           (void*)&global_counter,
           (void*)&local);
```

```
    return NULL;
}

int main(void) {
    pthread_t a, b;
    pthread_create(&a, NULL, thread_func, NULL);
    pthread_create(&b, NULL, thread_func, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    return 0;
}
```

> `pthread_self` returns the thread id if the calling thread.

This output will be similar to the following:

```
tid=139739974829824 global@0x55bc543d404c local@0x7f17bf8f7eec
tid=139739966437120 global@0x55bc543d404c local@0x7f17bf0f6eec
```

4 Returning Results from Threads

Note that the thread functions are declared to return a pointer to void. However, there are some pitfalls involved in using that pointer. The code below shows one attempt at returning an integer status code from a thread function.

```
// Might be a dangerous method.
void *thread_func(void *arg) {
    int code = 5;
    return (void *)code ;
}
```

This method will only work on machines where integers can be converted to a pointer and then back to an integer without losing information. On some machines, such conversions are dangerous. This method will fail whenever one attempts to return an object, such as a structure, that is larger than a pointer. In contrast, the code below does not fight the type system and returns a pointer to an internal buffer where the return value is stored. While the example shows an array of characters for the buffer, one can easily imagine it being an array of any necessary type or a single object, such as an integer status code or a structure with many members.

```
// This method will fail.
void *thread_func(void *arg) {
    char buffer[64] = "Hello world!";
    return buffer;
}
```

The code above fails because the internal buffer is automatically allocated and is deallocated as soon as the thread function returns. The pointer, which is given back to the calling thread, points at undefined memory. This is another example of the classic dangling pointer error. In the next attempt, the buffer is made `static` so it continues to exist even after the thread function terminates. This addresses the dangling-pointer problem.

```c
// This method has an overwriting problem in a multi-threaded program.
void *thread_func(void *arg) {
    static char buffer[64] = "Hello world!";
    return buffer;
}
```

This method might be satisfactory in some cases, but it does not work in the common case of multiple threads running the same thread function. In such a situation, the second thread will overwrite the static buffer with its own data and destroy the data left by the first thread. Global data suffers from this same problem since global data always has a static duration. The version below is the most general and robust.

```c
// The most robust returning method.
void *thread_func(void *arg) {
    char *buffer = malloc(64);
    strcpy(buffer, " Hello world!");
    return buffer;
}
```

This version allocates buffer space dynamically. This approach will work correctly even if multiple threads execute the thread function. Each will allocate a different array and store the address of that array in a stack variable. Every thread has its own stack, so automatic data objects are different for each thread. In order to receive the return value of a thread, the higher-level thread must join with the subordinate thread.

```c
void *threadresult;

// Wait for the thread to terminate.
pthread_join(threadID, &threadresult);
```

The `pthread_join()` function blocks until the thread specified by its first argument terminates. It then stores into the pointer pointed at by its second argument the value returned by the thread function. To use this pointer, the higher-level thread must cast it into an appropriate type and dereference it.

```c
char *message ;
message = (char *)threadresult ;
printf("I got %s back from the thread.\ n", message);
free(threadresult);
```

If the thread function dynamically allocates the space for the return value, then the higher-level thread needs to free that space when it no longer needs the return value. If this is not done, the program will leak memory. Below, you can see a complete code of returning results from threads.

```c
struct thread_args {
    int a;
    double b;
};

struct thread_result {
```

```c
    long x;
    double y;
};

void *thread_func(void *args) {
    struct thread_args * thread_args = args;
    struct thread_result *res = malloc(sizeof *res);
    res->x = 10 + thread_args->a;
    res->y = thread_args->a * thread_args->b;
    return res;
}

int main() {
    pthread_t threadL;
    struct thread_args in = {.a = 10, .b = 3.141592653};
    void *out_void;
    struct thread_result *out;
    pthread_create(&threadL, NULL, thread_func, &in);
    pthread_join(threadL, &out_void);
    out = out_void;
    printf("out -> x = %ld\tout -> b = %f\n", out->x, out->y);
    free(out);
    return 0;
}
```

5 Data Races

In this section, you will see how accessing shared data by multiple threads can produce unexpected results. In the following program, eight (`THREADS`) threads concurrently access a shared variable counter and increment it for `ITERS` times. The final result of this variable can be different than what is expected (`ITERS` * `THREADS`).

```c
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

#define THREADS 8
#define ITERS   1000000

static long counter = 0;

static void* worker(void* arg) {
    (void)arg;
    for (int i = 0; i < ITERS; i++) {
        counter++; // data race
    }
    return NULL;
}

int main(void) {
    pthread_t t[THREADS];
```

```

    for (int i = 0; i < THREADS; i++) pthread_create(&t[i], NULL, worker, NULL);
    for (int i = 0; i < THREADS; i++) pthread_join(t[i], NULL);

    printf("Expected: %ld\n", (long)THREADS * ITERS);
    printf("Actual:   %ld\n", counter);
    return 0;
}
```

We might get an output similar to the following:

```
Expected: 8000000
Actual:   1538398
```

The reason for unexpected results is that `counter++` is not atomic and contains `load` → `add` → `store` steps. The threads can interleave and do not wait for each other's updates. This happens when a thread has loaded the `counter` and wants to update it, before it updates, another thread loads the `counter` and tries to update it. So the updates are not performed effectively. This can happen on both single-core and multi-core CPU systems.

6 Process Creation vs Thread Creation Time

In this section, we will compare process and thread creation times using an example to experimentally assess thread creation efficiency.

6.1 Timing Basics

For measuring the execution time, we will use a standard high-resolution clock in a POSIX system using `time.h`:

```
clock_gettime()
```

We will use the following function to calculate the time elapsed in an interval in seconds:

```
double time_diff(struct timespec start, struct timespec end) {
    return (end.tv_sec - start.tv_sec) +
           (end.tv_nsec - start.tv_nsec) / 1e9;
}
```

6.2 Measuring Process Creation

In this section, we will create many short-lived processes that terminate quickly to measure only creation times.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>
```

```c

#define N 1000

double time_diff(struct timespec start, struct timespec end) {
    return (end.tv_sec - start.tv_sec) +
           (end.tv_nsec - start.tv_nsec) / 1e9;
}

int main() {

    struct timespec start, end;

    clock_gettime(CLOCK_MONOTONIC, &start);

    for(int i = 0; i < N; i++) {

        pid_t pid = fork();

        if(pid == 0) {
            exit(0);   // child exits immediately
        }
        else {
            wait(NULL);   // parent waits for child
        }
    }

    clock_gettime(CLOCK_MONOTONIC, &end);

    printf("Fork time for %d processes: %f seconds\n",
           N, time_diff(start,end));

    return 0;
}
```

Now compile this file and execute it considering various process numbers (100, 1000 and 10000).

```
gcc proc_create_time.c -o proc_create_time
./proc_create_time
```

6.3 Measuring Thread Creation

Now we will measure the creation time of multiple threads that are executed in the same process, sharing the same address space.

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <time.h>

#define N 1000
```

```c

void* thread_func(void *arg) {
    return NULL;
}

double time_diff(struct timespec start, struct timespec end) {
    return (end.tv_sec - start.tv_sec) +
           (end.tv_nsec - start.tv_nsec) / 1e9;
}

int main() {

    pthread_t thread;

    struct timespec start, end;

    clock_gettime(CLOCK_MONOTONIC, &start);

    for(int i = 0; i < N; i++) {
        pthread_create(&thread, NULL, thread_func, NULL);
        pthread_join(thread, NULL);
    }

    clock_gettime(CLOCK_MONOTONIC, &end);

    printf("Thread creation time for %d threads: %f seconds\n",
           N, time_diff(start,end));

    return 0;
}
```

Now we can execute this file under different thread counts (100, 1000, 10000).

```
gcc thread_create_time.c -pthread -o thread_create_time
./thread_create_time
```

6.4 Comparison Results

You will probably observe around a 10x speed-up in thread creation compared with process creation, depending on the operating system and hardware.

The reasons for the shorter creation time of a thread by the kernel are that creating a process requires setting up a new address space, creating page tables, duplicating file descriptors, and allocating process control structures. On the other hand, creating a thread only requires allocating a stack, creating a thread control block, and registering it with the scheduler, while reusing the existing process resources.

7 Thread Pools

Creating a new pthread for every tiny task is expensive, and a thread pool fixes this by i) creating N worker threads once, ii) keeping them alive, and iii) feeding them work from a shared job queue. In this section, we will use the cthreadpool[1] library to create and manage a pool of worker threads for assigning and executing jobs concurrently. Refer to the GitHub repository of this library for more detailed instructions.

7.1 Build and link

Compile your program and cthreadpool.c and link pthreads using the following shell command:

```
gcc -o your_program your_program.c cthreadpool.c -pthread
```

You can provide build scripts for CMake projects using the instructions in the repository.

7.2 Step-by-step thread pool creation

Now you will create a number of threads using this library and assign jobs to these threads.

7.2.1 Initialize pool

You can create a pool of threads using the following code:

```
threadpool_t *tp = threadpool_t_init(4);
```

This code creates a thread pool with four threads, a job queue, and a mutex/condition variable infrastructure.

7.2.2 Submit jobs

The following code allocates a job and pushes it into the queue under a mutex:

```
threadpool_add_work(tp, worker_func, arg_ptr);
```

After executing this code, the workers check the job queue and, if they find a job, execute it and free it. Otherwise, they will wait.

7.2.3 Wait for completion

With the following code, the program blocks until the job queue length and "threads working" count is zero:

```
threadpool_wait(tp);
```

7.2.4 Destroy

Now you can clean up resources by destroying the thread pool when done.

```
threadpool_destroy(tp);
```

[1] https://github.com/neo2043/cthreadpool

7.3 A "Hello jobs" example

Now we will create a pool of threads that print their job id and the thread ID that is executing the job.

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include "cthreadpool.h"

typedef struct {
    int job_id;
    threadpool_t *tp;
} job_arg_t;

void *print_job(void *arg) {
    job_arg_t *a = (job_arg_t*)arg;

    usleep(50 * 1000);

    printf("Job %d executed on worker thread id=%d\n",
           a->job_id, get_thread_id(a->tp));

    free(a);
    return NULL;
}

int main(void) {
    threadpool_t *tp = threadpool_t_init(4);

    for (int i = 0; i < 10; i++) {
        job_arg_t *a = malloc(sizeof(*a));
        if (!a) exit(1);
        a->job_id = i;
        a->tp = tp;
        threadpool_add_work(tp, print_job, a);
    }

    threadpool_wait(tp);
    threadpool_destroy(tp);
    return 0;
}
```

> The program consists of the following components:
>
> `job_arg_t`: data structure for input argument of thread functions;
>
> `print_job`: the function each thread executes;
>
> `get_thread_id`: is a function in the cthreadpool library and returns the ID of the worker thread that is currently executing the job.

After executing this code, you can see that each thread might execute multiple jobs as follows:

```
Job 3 executed on worker thread id=1
Job 0 executed on worker thread id=2
Job 2 executed on worker thread id=3
Job 1 executed on worker thread id=0
Job 4 executed on worker thread id=1
Job 5 executed on worker thread id=2
Job 6 executed on worker thread id=3
Job 7 executed on worker thread id=0
Job 8 executed on worker thread id=1
Job 9 executed on worker thread id=2
```

8 Thread Cancellation

Thread cancellation is a mechanism that allows one thread to request the termination of another thread. The function used for thread cancellation is:

```
pthread_cancel(thread);
```

This function sends a cancellation request and terminates the thread immediately or deferred, depending on the cancellation type. You can check the documentation of this function with:

```
man pthread_cancel
```

8.1 Thread Cancellation Settings

There are two kinds of settings for thread cancellation:

8.1.1 Cancelability state

We can enable thread cancelability by other threads within each thread with:

```
pthread_setcancelstate(PTHREAD_CANCEL_ENABLE, NULL);
```

or disable it with:

```
pthread_setcancelstate(PTHREAD_CANCEL_DISABLE, NULL);
```

8.1.2 Cancelability type

We can set the thread cancelability type within a thread to be deferred with:

```
pthread_setcanceltype(PTHREAD_CANCEL_DEFERRED, NULL);
```

or to be of type asynchronous with:

```
pthread_setcanceltype(PTHREAD_CANCEL_ASYNCHRONOUS, NULL);
```

8.2 Deferred Cancellation

This is the default and the safe cancellation method. The thread is canceled only when it reaches a cancellation point. Typical cancellation points include functions like: `sleep()`, `read()`, `pthread_join()`, `pthread_cond_wait()`, `pause()` that a canellation can occur. You can check the complete list in the manual:

```
man pthreads
```

You can also create a cancellation point manually using:

```
pthread_testcancel();
```

The thread keeps running until it reaches one of those cancellation points. If there is a pending cancellation request, it will be acted on. In the following example code, we will apply a deferred cancellation on a thread by setting cancellation settings and implicit and explicit cancellation points.

```c
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>
#include <time.h>

void *worker(void *arg) {
    printf("Worker (deferred): started\n");
    pthread_setcancelstate(PTHREAD_CANCEL_ENABLE, NULL);
    pthread_setcanceltype(PTHREAD_CANCEL_DEFERRED, NULL);
    while (1) {
        printf("Worker: working (no cancellation point)...\n");
        // Busy work (no cancellation point)
        for (volatile long i = 0; i < 10000000000; i++);
        printf("Worker: reaching the cancellation point\n");
        pthread_testcancel(); //<-- cancellation happens here (or we can use ↩
            sleep(2);)
    }
    return NULL;
}

int main(void) {
    pthread_t tid;
    void *retval;
    pthread_create(&tid, NULL, worker, NULL);
    sleep(1);
    printf("Main: sending cancellation request (deferred)\n");
    pthread_cancel(tid);
    printf("Main: waiting...\n");
    pthread_join(tid, &retval);
    if (retval == PTHREAD_CANCELED) {
        printf("Main: thread canceled (deferred)\n");
    }
    return 0;
}
```

> Main creates the thread with `pthread_create(&tid, NULL, worker, NULL)`.
>
> Inside worker, cancellation is enabled with `pthread_setcancelstate(PTHREAD_CANCEL_ENABLE, NULL)`.
>
> The thread chooses deferred mode with `pthread_setcanceltype(PTHREAD_CANCEL_DEFERRED, NULL)`.
>
> The worker runs a loop with a long for loop inside it and periodically reaches cancellation points of `pthread_testcancel()`.
>
> After 1 second, main requests cancellation with `pthread_cancel(tid)`. This does not necessarily stop the thread instantly.
>
> The worker is canceled when it next reaches a cancellation point (after the long for loop).
>
> Main waits for termination using `pthread_join(tid, &retval)`
>
> If the thread was canceled, `retval == PTHREAD_CANCELED`.

The output of this code will be similar to the following:

```
Worker (deferred): started
Worker: working (no cancellation point)...
Main: sending cancellation request (deferred)
Main: waiting...
Worker: reaching the cancellation point
Main: thread canceled (deferred)
```

8.3 Asynchronous Cancellation

Asynchronous cancellation represents the most unsafe cancellation mode. In this mode, a thread may be terminated at virtually any point during its execution, for example, while holding a mutex, modifying shared data structures, performing I/O operations, and executing within a critical section. Such abrupt termination can violate invariants, leave shared resources in inconsistent states, and lead to deadlocks, data corruption, or resource leaks. It provides low-latency termination but at the cost of safety and predictability. For this reason, it is generally discouraged in practical systems and should only be used in carefully controlled scenarios.

In the following code, we can see an example of asynchronous cancellation:

```c
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

void *worker(void *arg) {
    printf("Worker (async): started\n");
    pthread_setcancelstate(PTHREAD_CANCEL_ENABLE, NULL);
    pthread_setcanceltype(PTHREAD_CANCEL_ASYNCHRONOUS, NULL);
    while (1) {
```

```c
        printf("Worker: working...\n");
        // Same busy work
        for (volatile long i = 0; i < 10000000000; i++);
        printf("Worker: about to sleep\n");
        sleep(2);  // may never reach this
    }
    return NULL;
}

int main(void) {
    pthread_t tid;
    void *retval;
    pthread_create(&tid, NULL, worker, NULL);
    sleep(1);
    printf("Main: sending cancellation request (async)\n");
    pthread_cancel(tid);
    printf("Main: waiting...\n");
    pthread_join(tid, &retval);
    if (retval == PTHREAD_CANCELED) {
        printf("Main: thread canceled immediately (async)\n");
    }
    return 0;
}
```

> The thread enables cancellation and sets the type to asynchronous with `pthread_setcanceltype(PTHREAD_CANCEL_ASYNCHRONOUS, NULL)`
>
> Main sends a cancellation request with `pthread_cancel(tid)` and the worker is terminated almost immediately and does not wait to finish the long for loop.

The output of this code is similar to the deferred cancellation example. The difference is in the cancellation time.

9 fork and exec with Multi-threading (Optional)

In this section, you will understand what happens when a process uses fork and multi-threading together, and you will learn the safe way to do it.

9.1 fork After Creating Threads

In the following code, the main process creates a thread (executing the worker thread function) and then creates a child process.

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>
#include <sys/types.h>
#include <sys/wait.h>

void *worker(void *arg) {
    while (1) {
        printf("Worker thread running in process %d\n", getpid());
        sleep(1);
    }
    return NULL;
}

int main() {
    pthread_t tid;

    if (pthread_create(&tid, NULL, worker, NULL) != 0) {
        perror("pthread_create");
        return 1;
    }

    sleep(2);  // let worker run

    printf("Main thread in process %d calling fork()\n", getpid());
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    } else if (pid == 0) {
        printf("Child process %d after fork()\n", getpid());
        printf("Only the calling thread exists in the child.\n");
        sleep(3);
    } else {
        printf("Parent process %d continues, child=%d\n", getpid(), pid);
        sleep(3);
    }

    return 0;
}
```

In this program, the parent process has two threads (main and worker threads). After forking, the parent still has two threads, but the child has only one, i.e., the thread that called `fork()`, and the worker thread is not copied into the child. You should observe some output as below, where the worker output continues in the parent, but the child does not have that worker thread.

```
Worker thread running in process 18015
Worker thread running in process 18015
Main thread in process 18015 calling fork()
Parent process 18015 continues, child=18022
Worker thread running in process 18015
Child process 18022 after fork()
Only the calling thread exists in the child.
Worker thread running in process 18015
Worker thread running in process 18015
```

9.2 fork Before Creating Threads

Now in the following program, we will create a child process first and then create threads for both parent and child processes:

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>
#include <sys/types.h>
#include <sys/wait.h>

void *worker(void *arg) {
    printf("Thread in process %d is running\n", getpid());
    return NULL;
}

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    pthread_t tid;

    if (pid == 0) {
        printf("Child process %d created after fork()\n", getpid());

        if (pthread_create(&tid, NULL, worker, NULL) != 0) {
            perror("pthread_create");
            return 1;
        }
        pthread_join(tid, NULL);

```

```
        printf("Child process %d finished its thread\n", getpid());
    } else {
        printf("Parent process %d continues after fork()\n", getpid());

        if (pthread_create(&tid, NULL, worker, NULL) != 0) {
            perror("pthread_create");
            return 1;
        }
        pthread_join(tid, NULL);

        wait(NULL);
        printf("Parent process %d finished its thread\n", getpid());
    }

    return 0;
}
```

Executing this code will represent the execution of different threads inside the parent and child processes and gives an output similar to this:

```
Parent process 18535 continues after fork()
Thread in process 18535 is running
Child process 18536 created after fork()
Thread in process 18536 is running
Child process 18536 finished its thread
Parent process 18535 finished its thread
```

Creating threads after `fork()` is usually much cleaner than forking an already multithreaded process, as you will see in the later section.

9.3 fork and exec with Multi-Threading

In this section, you will apply `exec()` after creating the child process as the following code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>
#include <sys/types.h>
#include <sys/wait.h>

void *worker(void *arg) {
    while (1) {
        printf("Worker thread in parent process %d\n", getpid());
        sleep(1);
    }
    return NULL;
}

int main() {
    pthread_t tid;

    if (pthread_create(&tid, NULL, worker, NULL) != 0) {
```

```
        perror("pthread_create");
        return 1;
    }

    sleep(2);

    printf("Main thread calling fork()\n");
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    } else if (pid == 0) {
        printf("Child process %d calling exec()\n", getpid());
        execlp("ls", "ls", "-l", NULL);
        perror("execlp");
        exit(1);
    } else {
        wait(NULL);
        printf("Parent process %d finished waiting\n", getpid());
    }

    return 0;
}
```

In this program parent has two threads (main and worker). One thread calls `fork()` and a child is created with only the calling thread. Then the child immediately calls `exec()`, and the child's entire old program image is replaced by `ls`. You should get an output similar to this:

```
Worker thread in parent process 19503
Worker thread in parent process 19503
Main thread calling fork()
Worker thread in parent process 19503
Child process 19505 calling exec()
total 72
-rwxr-xr-x 1 haleh haleh 17112 Mar  9 19:39 fork_exec_thread
-rw-r--r-- 1 haleh haleh   867 Mar  9 19:37 fork_exec_thread.c
-rwxr-xr-x 1 haleh haleh 16968 Mar  9 19:15 fork_then_thread
-rw-r--r-- 1 haleh haleh  1043 Mar  9 19:14 fork_then_thread.c
-rwxr-xr-x 1 haleh haleh 16960 Mar  9 19:02 thread_then_fork
-rw-r--r-- 1 haleh haleh   916 Mar  9 19:01 thread_then_fork.c
Parent process 19503 finished waiting
```

This is good because the child does not keep running the half-broken thread state inherited from the `fork()` call (you will see in the next section). This is why `fork()` + `exec()` is the normal safe design.

9.4 Insecure fork After Creating Threads (Without exec)

Now you will see how using a fork without exec with multi-threading can be dangerous and result in a deadlock when using a mutex.

Consider the following program:

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>
#include <sys/types.h>
#include <sys/wait.h>

pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void *worker(void *arg) {
    pthread_mutex_lock(&lock);
    printf("Worker thread locked mutex in process %d\n", getpid());
    sleep(10);   // keep lock held
    pthread_mutex_unlock(&lock);
    return NULL;
}

int main() {
    pthread_t tid;

    if (pthread_create(&tid, NULL, worker, NULL) != 0) {
        perror("pthread_create");
        return 1;
    }

    sleep(1); // let worker lock mutex

    printf("Main thread calling fork()\n");
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    } else if (pid == 0) {
        printf("Child process trying to lock mutex...\n");
        pthread_mutex_lock(&lock);   // may deadlock
        printf("Child acquired mutex\n");
        pthread_mutex_unlock(&lock);
        exit(0);
    } else {
        wait(NULL);
        pthread_join(tid, NULL);
    }

    return 0;
}
```

In this code:

- **before fork()** worker thread locks mutex, main thread calls `fork()`;
- **after fork()** in the child, only the main thread exists, and the worker thread is gone.

However, the mutex may still be marked as locked, so the child may block forever trying to lock a mutex that can never be unlocked.

Execution of this program might result in this output:

```
Worker thread locked the mutex in process 21200
Main thread calling fork()
Child process trying to lock mutex...
```

Where a deadlock occurs, it will never terminate.